TRM Latent-History Project
Technical Handoff and Collaboration Protocol

Project Team

August 21, 2026

Contents

1 Purpose of This Document

This document is the technical handoff for our Deep Learning project based on Tiny Recursive Models (TRM).

The main research direction is to investigate whether a TRM can benefit from explicit access to its previous outer-step latent states.

The central proposed method is:

Selective Attention over TRM Latent History

The project has already established a reproducible CPU baseline, a controlled evaluation setup, a safe latent-history buffer, and a modular history aggregation interface.

From this point onward, the work is intentionally divided into two parallel research tracks:

1. **HistoryAttention track**: implemented primarily by the teammate who proposed the attention idea.
2. **Non-attention history baselines and controlled evaluation track**: implemented primarily by the current TRM/core owner.

The objective of this division is to preserve a stable TRM core, reduce merge conflicts, and allow a scientifically meaningful comparison between attention and simpler uses of latent history.

2 Repository and Current Handoff Point

Repository:

https://github.com/Iliyabr/trm-latent-history-attention

The shared handoff branch is:

`phase2-history-interface`

The current handoff commit is:

`af02f1b`

Commit message:

`Add pluggable history aggregator interface`

Important earlier checkpoints are:

- `14de0bb` — Finalize reproducible Phase 1 baseline study
- `413e67f` — Add validated latent-history interface
- `af02f1b` — Add pluggable history aggregator interface

The working tree was clean immediately after the final handoff commit.

The handoff commit should be treated as the common starting point for the two parallel branches.

3 Research Question

The original TRM recursively updates latent states during reasoning.

The proposed extension asks whether the current reasoning state should be able to selectively retrieve information from previous outer reasoning states.

At outer reasoning step t, define the previous latent history as:

$$\mathcal{H}_t = \{z_1, z_2, \ldots, z_{t-1}\}.$$

The proposed HistoryAttention method should use the current state z_t as a query and previous states as candidate memory.

A generic formulation is:

$$Q_t = f_Q(z_t),$$

$$K_i = f_K(z_i), \qquad V_i = f_V(z_i), \qquad i < t,$$

followed by selective retrieval:

$$c_t = \text{Attention}\left(Q_t, K_{<t}, V_{<t}\right).$$

The retrieved context c_t can then be fused with the current latent state.

The exact attention design has intentionally **not** been fixed yet. That design is part of the HistoryAttention research contribution.

4 Important Scientific Distinction

The original TRM implementation already contains attention inside the reasoning module.

However, this is not the same as the proposed HistoryAttention mechanism.

The distinction is:

- **Original TRM attention**: intra-state or token-level self-attention inside a single latent state.
- **Our proposed attention**: inter-step selective retrieval over latent states from previous outer reasoning steps.

Therefore, the proposed contribution should not be described merely as "adding attention to TRM."

A more accurate description is:

> Selective retrieval over recursive latent-state history.

5 Completed Work

5.1 Phase 0: Reproducible CPU Foundation

A CPU-only development workflow was established.

The validated environment included approximately:

- Python 3.12
- PyTorch 2.7 CPU build
- CUDA disabled

- CPU thread count fixed to 8

A very small Sudoku smoke dataset was also used to validate training, checkpointing, model loading, and inference.

This phase was intended only for infrastructure verification, not scientific claims.

5.2 Phase 1: Reproducible Scientific Baseline

A deterministic Sudoku baseline dataset was created at:

`data/sudoku-baseline-v2`

The dataset builder is:

`dataset/build_sudoku_baseline_v2.py`

The corresponding manifest is:

`artifacts/data/sudoku_baseline_v2_manifest.json`

The source dataset was:

`sapientinc/sudoku-extreme`

The controlled development split was:

- 1000 original training puzzles
- 200 original development puzzles
- train/dev split performed before augmentation
- 10 augmentations per training puzzle
- no augmentation on development puzzles
- deterministic random seed equal to 0
- official test data not used during development

The canonical baseline configuration is:

`config/cfg_baseline_v2.yaml`

The experiment documentation is located in:

`experiments/baseline/README.md`

and

`experiments/baseline/PHASE1_RESULTS.md`

A baseline regression test is located at:

`tests/test_baseline_regression.py`

5.3 Phase 1 Evaluation Infrastructure

The training script was extended so that development evaluation can be selected explicitly.

Important infrastructure includes:

- configurable evaluation split
- JSON metric output
- token/cell accuracy
- exact puzzle accuracy
- language-model loss
- halt-related metrics
- reasoning-step metrics

The official Sudoku test split remains untouched during model development and hyperparameter selection.

5.4 Controlled Baseline Results

At a matched 40-epoch optimization budget, the reduced CPU TRM produced:

Configuration	Dev Accuracy	LM Loss
TRM-2	48.247%	1.21854
TRM-4	45.840%	1.35791
TRM-8	42.815%	1.44462
TRM-16	42.173%	1.46285

All exact-puzzle accuracies in this reduced development regime were zero.

Therefore, these results should be interpreted as development evidence based on cell/token accuracy and loss, not as final Sudoku-solving results.

The defensible conclusion is:

> Under the reduced CPU regime and a fixed 40-epoch optimization budget, increasing outer reasoning depth alone did not improve development performance.

It would be incorrect to claim that recursion is intrinsically harmful.

5.5 TRM-4 Controlled Hyperparameter Sweep

The primary matched-history development configuration was selected as TRM-4.

The controlled sweep used:

- halt max steps = 4
- epochs = 40
- seed = 0
- batch size = 4
- hidden size = 64
- learning rate = 10^{-3}

- L cycles = 1

Selected results were:

Variant	Accuracy	LM Loss
Reference	45.840%	1.357913
LR 3×10^{-4}	45.630%	1.370709
LR 3×10^{-3}	42.870%	1.432477
Hidden 96	42.877%	1.411830
Hidden 128	43.049%	1.411394
L cycles = 2	43.444%	1.402203

Within this controlled search space, the reference configuration remained best.
The sweep summary is stored at:

`results/baseline/trm4-40ep-sweep/tuning_results.json`

The tuning runner is:

`scripts/run_trm4_40ep_tuning.py`

The parameter counter is:

`scripts/count_trm_params.py`

6 Why TRM-4 Is the Main History Baseline

TRM-4 was selected as the primary matched-history development baseline.
The reasoning is:

- TRM-2 provides too little history for a meaningful history-selection experiment.
- TRM-8 and TRM-16 degrade more strongly in the reduced CPU regime.
- Using a strongly degraded model would create a confound: a history mechanism might appear useful simply because it repairs over-recursion damage.
- TRM-4 provides non-trivial latent history while remaining reasonably competitive.

Therefore:

- TRM-2 is the performance-reference baseline.
- TRM-4 is the primary HistoryAttention comparison point.
- TRM-8 is a longer-history stress test.
- TRM-16 is a paper-aligned maximum-step stress test.

7 Phase 2-A: Validated Latent-History Buffer

The TRM core was extended with a recording-only latent-history interface.

The main modified file is:

`models/recursive_reasoning/trm.py`

The history safety tests are in:

`tests/test_history_interface.py`

The history is based on the outer-step latent state z_H.
This state was selected because it is:

- carried between outer reasoning steps;
- used to produce the language-model output;
- explicitly detached across outer reasoning steps in the original TRM implementation.

7.1 History Storage

The carry now optionally contains:

- history_z_H
- history_lengths

The history tensor has shape:

$$[B, K, L, D],$$

where:

- B = batch size
- K = maximum outer reasoning steps
- L = sequence length
- D = hidden dimension

For the current TRM-4 baseline:

$$[B, K, L, D] = [4, 4, 81, 64].$$

7.2 Causal Semantics

Before outer step 1:

$$\mathcal{H}_1 = \varnothing.$$

After step 1:

$$\mathcal{H} = [z_1].$$

Before step 2:

$$\mathcal{H}_2 = [z_1].$$

After step 2:

$$\mathcal{H} = [z_1, z_2].$$

Therefore a future HistoryAttention module must only consume strictly previous states.

The current state must never attend to itself through the history buffer.

7.3 Gradient Policy

Stored history states are detached.

Conceptually:

$$\bar{z}_t = \text{stopgrad}(z_t).$$

This preserves the original truncated-gradient behavior across outer reasoning steps and avoids full backpropagation through the entire latent trajectory.

This is especially important in the current CPU regime.

If gradient-through-history is investigated later, it must be treated as a separate ablation rather than silently changing the training semantics.

7.4 Reset Safety

History is reset independently for each batch slot when that slot is reused for a new puzzle.

This prevents latent-history leakage from one Sudoku puzzle into another.

The reset-isolation test explicitly verifies mixed batches where some samples continue and other samples reset.

7.5 Validated Phase 2-A Invariants

The following properties have been tested:

- history tensor has the expected shape;
- history length increases correctly;
- stored states are detached;
- the newest stored state equals the detached current state;
- reset slots do not retain previous-puzzle history;
- continuing slots preserve their previous history;
- enabling recording-only history leaves vanilla logits exactly unchanged.

The last property is checked with exact tensor equality rather than approximate floating-point equality.

8 Phase 2-B: Pluggable History Aggregator Interface

A modular history aggregation API was added so that future methods can be implemented outside the TRM recursion core.

The new directory is:

`models/history/`

It currently contains:

- `models/history/base.py`
- `models/history/none.py`
- `models/history/factory.py`
- `models/history/__init__.py`
- `models/history/README.md`

8.1 Shared Aggregator Contract

Every history aggregator receives:

$$\text{current\_z} \in \mathbb{R}^{B\times L\times D},$$

$$\text{history\_z} \in \mathbb{R}^{B\times K\times L\times D},$$

and

$$\text{history\_lengths} \in \mathbb{N}^{B}.$$

It must return:

$$\text{updated\_z} \in \mathbb{R}^{B\times L\times D}.$$

Conceptually, the API is:

```
updated_z = aggregator(
    current_z=current_z,
    history_z=history_z,
    history_lengths=history_lengths,
)
```

The output shape and dtype must match current_z.

8.2 Integration Order

The integration order is intentionally:

1. compute current z_H;
2. call the history aggregator using strictly previous history;
3. obtain updated z_H;
4. detach the updated state;
5. append it to the history buffer;
6. produce/carry model outputs.

This ordering is critical.

It guarantees that a history method cannot accidentally read the current state from the history buffer.

8.3 NoHistoryAggregator

A parameter-free identity implementation already exists.

It conceptually performs:

```
return current_z
```

It:

- ignores history;
- introduces zero trainable parameters;
- introduces no arithmetic transformation;
- preserves vanilla TRM behavior.

The corresponding test verifies that the model parameter count is unchanged.

9 Current Tests That Must Continue to Pass

The existing history-related tests can be run with:

```
python -c "import tests.test_history_interface as t; t.test_history_interface_cpu(); t.
    test_history_reset_isolation_cpu(); t.test_no_history_aggregator_contract_cpu()"
```

The expected successful messages are:

```
P2-A HISTORY INTERFACE PASS
P2-A HISTORY RESET ISOLATION PASS
P2-B NO-HISTORY CONTRACT PASS
```

The Phase-1 regression can be run with:

```
python -c "import runpy; ns=runpy.run_path('tests/test_baseline_regression.py'); ns['
    test_baseline_checkpoint_and_forward_cpu']()"
```

Expected result:

```
P1-D BASELINE REGRESSION PASS
```

Before committing code, also run:

```
git diff --check
```

and after staging:

```
git diff --cached --check
```

No output from these Git checks means no whitespace errors were detected.

10 Team Work Division From This Point

10.1 Teammate: HistoryAttention Owner

The teammate who proposed the HistoryAttention idea should own the main attention design.

Primary responsibilities:

1. design the HistoryAttention mechanism;
2. define query, key, and value construction;
3. correctly mask invalid history positions;

4. handle samples with zero valid history;
5. choose the fusion rule between retrieved history and current state;
6. implement HistoryAttention;
7. write attention-specific unit tests;
8. report parameter overhead;
9. later analyze attention weights if scientifically useful;
10. document the mathematical formulation.

Preferred files to modify:

- `models/history/attention.py`
- `models/history/factory.py`
- `tests/test_history_attention.py`

The teammate should first read:

`models/history/README.md`

before implementing the module.

10.2 Current TRM/Core Owner: Non-Attention Track

The other development track should implement simpler history methods that test whether attention is actually necessary.

Primary planned methods are:

1. UniformMeanHistory
2. RecencyWeightedHistory
3. GatedHistory

The scientific role of these methods is critical.
They test whether improvements come from:

having access to history at all

versus:

selectively retrieving relevant history using learned attention.

This track also owns:

- shared evaluation scripts;
- parameter-count comparisons;
- controlled ablations;
- multi-seed aggregation later;
- final comparison tables and plots;
- integration consistency.

11 Scientific Comparison Structure

The intended final comparison is approximately:

Model	Scientific Role
Vanilla TRM	No history
Uniform Mean	History without selection
Recency Weighted	Fixed temporal preference
Gated History	Learned selection without full attention
HistoryAttention	Learned selective retrieval

This structure allows us to test two related hypotheses.

Hypothesis 1

H_1 : Explicit latent history can improve recursive reasoning.

Hypothesis 2

H_2 : Selective attention over history provides value beyond simple history aggregation.

HistoryAttention should therefore not be evaluated only against vanilla TRM.
It should also be compared against simpler history aggregators.

12 Files That Should Be Treated as Stable

The following files or systems have already been validated and should not be modified casually:

- `models/recursive_reasoning/trm.py`
- `pretrain.py`
- `dataset/build_sudoku_baseline_v2.py`
- `config/cfg_baseline_v2.yaml`
- `tests/test_baseline_regression.py`
- Phase-1 dataset split and manifest
- Phase-1 recorded baseline results

In particular, the teammate implementing HistoryAttention should avoid editing:

`models/recursive_reasoning/trm.py`

unless the shared history interface is genuinely insufficient.

13 Protocol for Changing Validated Core Code

If a future method appears to require a change to validated core code, do not immediately edit the core implementation.

Use the following procedure.

1. First determine whether the feature can be implemented entirely inside the history module.
2. If not, document exactly what information is missing from the current HistoryAggregator interface.
3. Propose the smallest possible interface change.
4. Discuss the interface change with the other developer before editing the TRM recursion logic.
5. Add or update a regression test that captures the intended invariant.
6. Make the minimal implementation change.
7. Re-run all Phase 1 and Phase 2 regression tests.
8. Verify that the NoHistory path remains exactly equivalent to the validated vanilla path.
9. Commit the interface/core change separately from the experimental method implementation whenever possible.

A core change should never be mixed into a large experimental commit without tests.

14 Important HistoryAttention Design Constraints

The HistoryAttention implementation should satisfy all of the following.

14.1 Causality

At step t, only states with indices less than t may be consumed.

$$z_t \notin \mathcal{H}_t.$$

14.2 Variable History Length

Different batch samples may have different valid history lengths.

The implementation must therefore use history_lengths rather than assuming all K entries are valid.

14.3 Zero-History Case

At the first outer step:

$$|\mathcal{H}_1| = 0.$$

The module must define safe behavior for this case.

The output must remain finite and well-defined.

14.4 No Cross-Puzzle Leakage

History entries belonging to a previous puzzle must never be visible after that batch slot resets.

14.5 Detached History

Previous history states are currently detached.

Do not silently re-enable gradient flow through historical states.

14.6 Output Compatibility

The aggregator must return a tensor with the same shape as current_z:

$$[B, L, D].$$

14.7 Numerical Safety

The implementation must avoid NaN values, particularly when all history positions are masked.

14.8 Parameter Accounting

The number of additional trainable parameters introduced by HistoryAttention must be reported.

Later, if possible, a parameter-matched vanilla control should be considered.

15 HistoryAttention Questions That Are Intentionally Open

The following design decisions belong to the HistoryAttention research track and have intentionally not been predetermined:

- whether attention is performed independently for every token;
- whether history is pooled before attention;
- whether queries come directly from z_t or from a projection;
- whether keys and values use separate projections;
- whether attention is single-head or multi-head;
- whether attention operates across outer steps only or across both step and token dimensions;
- how context is fused with current state;
- whether a learnable gate should control history contribution;
- whether normalization is required before or after fusion.

These decisions should be made deliberately and documented.

Avoid unnecessary architectural complexity during the first implementation.

The first HistoryAttention version should preferably be the smallest mechanism that directly tests the research hypothesis.

16 Recommended Branch Workflow

Both developers should branch from the common handoff commit.

The HistoryAttention developer can use:

```
git checkout phase2-history-interface
git pull origin phase2-history-interface
git checkout -b feature/history-attention
```

The non-attention developer can use:

```
git checkout phase2-history-interface
git pull origin phase2-history-interface
git checkout -b feature/history-baselines
```

Each developer should commit small, logically isolated changes.

Examples:

```
git add models/history/attention.py
git add models/history/factory.py
git add tests/test_history_attention.py
git commit -m "Add causal latent-history attention"
```

For non-attention methods, separate commits are preferable, for example:

```
git commit -m "Add uniform history baseline"
git commit -m "Add recency-weighted history baseline"
git commit -m "Add gated history baseline"
```

Avoid combining unrelated architectural changes, experiments, plots, and dataset changes into one commit.

17 Merge Protocol

Before asking the other developer to merge a branch:

1. ensure the working tree is clean;
2. run the method-specific tests;
3. run the complete Phase 2 history tests;
4. run the Phase 1 baseline regression;
5. run Git whitespace checks;
6. provide the exact commit hash;
7. provide a short description of files changed;
8. document any new configuration fields.

The developer responsible for TRM/core integration should perform the final merge when changes affect the shared model path.

This reduces the risk of two people modifying the recursion logic independently.

18 Acceptance Criteria for the First HistoryAttention Implementation

The first HistoryAttention implementation should not be considered complete until the following conditions hold:

1. It implements the shared HistoryAggregator interface.
2. It does not require direct modification of TRM recursion logic unless absolutely necessary.
3. It correctly handles history length zero.
4. It correctly masks unused history slots.

5. It only consumes strictly previous states.
6. Its output shape equals the input current-state shape.
7. It produces finite outputs.
8. It has a dedicated unit test.
9. Reset isolation still passes.
10. The existing NoHistory equivalence test still passes.
11. Phase-1 baseline regression still passes.
12. The additional parameter count is reported.
13. A minimal CPU forward pass succeeds.

Only after these conditions pass should training experiments begin.

19 Recommended First HistoryAttention Experiment

Do not immediately launch a large sweep.
First validate one controlled configuration:

- baseline architecture: TRM-4
- epochs: 40
- seed: 0
- batch size: 4
- hidden size: 64
- learning rate: 10^{-3}
- L cycles: 1
- same development split
- same training data
- same evaluation pipeline

The purpose of the first run is not to claim statistical significance.
It is to answer:

> Does the HistoryAttention implementation train correctly and produce a meaningful signal under the exact same reduced development regime?

Only after the implementation is validated should multiple seeds and broader ablations be run.

20 Planned Non-Attention Methods

20.1 Uniform Mean History

A simple baseline can compute:

$$\bar{h}_t = \frac{1}{t-1} \sum_{i=1}^{t-1} z_i.$$

This tests whether simply exposing accumulated historical information is enough.

20.2 Recency-Weighted History

A fixed weighting scheme can prefer recent states:

$$w_i \propto \exp(-\lambda(t-i)).$$

Then:

$$h_t = \sum_{i<t} w_i z_i.$$

This tests whether a simple temporal prior can explain any benefit.

20.3 Gated History

A lightweight learned gate can combine current state and aggregated history:

$$g_t = \sigma\left(W_g[z_t; h_t]\right),$$

$$z'_t = g_t \odot z_t + (1-g_t) \odot h_t.$$

This provides a learned selection mechanism without full HistoryAttention.

21 Experimental Discipline

The following rules should be maintained throughout the project.

1. Do not tune on the official test set.
2. Keep the train/dev split fixed.
3. Use the same evaluation pipeline across all compared models.
4. Avoid changing multiple hyperparameters at the same time during controlled ablations.
5. Record exact configuration values for every meaningful run.
6. Record parameter counts.
7. Record random seeds.
8. Distinguish development experiments from final paper experiments.
9. Do not make general claims from seed 0 alone.
10. Run multiple seeds before final statistical claims.
11. Do not claim that deeper recursion is generally harmful based only on the reduced CPU regime.
12. Do not claim that more epochs do not help; the existing experiments already show that some deeper configurations improve with additional optimization.

22 Recommended Instructions for an AI Coding Assistant

The following prompt can be given to an AI coding assistant before working on the HistoryAttention branch.

```
You are helping implement a research extension of Tiny Recursive Models
(TRM) called latent-history aggregation.

Before changing any code, follow these project rules:

1. The common validated starting point is commit af02f1b on branch
   phase2-history-interface.

2. Read models/history/README.md first.

3. The TRM core already records detached causal latent history.

4. The shared aggregator interface is:

   current_z:
       [B, L, D]

   history_z:
       [B, K, L, D]

   history_lengths:
       [B]

   output:
       [B, L, D]

5. At outer step t, the valid history must contain only states from
   steps 1 through t-1.

6. Never allow the current state to attend to itself through the
   history buffer.

7. Historical states are intentionally detached.
   Do not change this gradient policy unless explicitly asked to
   implement a separate ablation.

8. Batch samples can have different valid history lengths.
   Mask invalid history slots correctly.

9. The zero-history case must be numerically safe.

10. Avoid modifying
    models/recursive_reasoning/trm.py.
    Prefer implementing new behavior entirely inside models/history/.

11. For the HistoryAttention implementation, prefer changing only:
    models/history/attention.py
    models/history/factory.py
    tests/test_history_attention.py

12. If the existing interface is insufficient, stop and explain:
    - exactly what information is missing,
    - why HistoryAttention cannot be implemented through the existing API,
    - the smallest interface change required.
    Do not directly rewrite the TRM core.
```

```
13. Preserve the existing NoHistoryAggregator behavior.
    It is a zero-parameter identity control and must remain bitwise
    equivalent to vanilla TRM.

14. Do not change the dataset split, baseline configuration, or
    evaluation protocol.

15. Do not begin large training runs before unit tests and regression
    tests pass.

16. Implement the simplest scientifically meaningful HistoryAttention
    first. Avoid unnecessary architecture complexity.

17. Every architectural choice must be explainable scientifically:
    query definition, key/value definition, masking, attention axis,
    fusion rule, normalization, and parameter overhead.

18. After each implementation change, run the relevant regression tests.

19. Keep commits small and logically isolated.

20. Do not silently fix or refactor unrelated project code.

Your immediate task is to design and implement HistoryAttention as a
subclass of the existing HistoryAggregator interface while preserving
all validated project invariants.
```

23 Prompt for AI Before Modifying Validated Core Code

If the AI assistant proposes modifying the TRM core, use this prompt before accepting the change:

```
Before modifying validated TRM core code, perform an interface-impact
analysis.

Do not edit anything yet.

Explain:

1. Why the requested feature cannot be implemented entirely inside
   the existing HistoryAggregator abstraction.

2. Which exact tensor, metadata, or lifecycle information is missing.

3. The smallest API extension that would make the implementation
   possible.

4. Which existing invariants could be affected.

5. Which tests must be added or rerun.

6. Whether NoHistoryAggregator can remain exactly equivalent to the
   current vanilla behavior.

7. Whether old checkpoints remain load-compatible.

8. Whether the proposed change alters gradient flow.

9. Whether the proposed change alters history causality.
```

```
10. Whether the proposed change can cause cross-puzzle state leakage.

Only after this analysis should a minimal patch be proposed.
```

24 Prompt for AI Before Committing

Before committing a feature, the following prompt can be used:

```
Review the current git diff as a research-code integration review.

Check specifically for:

1. accidental changes outside the intended files;
2. modifications to TRM recursion semantics;
3. history causality violations;
4. invalid history masking;
5. zero-history numerical problems;
6. unintended gradient-through-history;
7. cross-puzzle history leakage;
8. checkpoint compatibility issues;
9. unintended parameter-count changes;
10. output shape or dtype changes;
11. changes to baseline data or evaluation protocol;
12. missing tests;
13. unnecessary refactoring;
14. hidden experimental confounds.

Do not rewrite the code unless a concrete issue is identified.

At the end, provide:
- files changed,
- invariants preserved,
- tests required before commit,
- remaining risks.
```

25 What Each Developer Should Return to the Other

When a development branch is ready for handoff, provide:

1. branch name;
2. latest commit hash;
3. list of changed files;
4. short architecture summary;
5. new configuration fields, if any;
6. parameter overhead;
7. tests executed;
8. exact test results;
9. any unresolved concerns;

10. whether core TRM code was modified.

For example:

```
Branch:
feature/history-attention

Commit:
<commit hash>

Changed files:
models/history/attention.py
models/history/factory.py
tests/test_history_attention.py

Core TRM modified:
No

Tests:
HistoryAttention unit test: PASS
History interface regression: PASS
Reset isolation: PASS
NoHistory equivalence: PASS
Phase-1 baseline regression: PASS

Additional parameters:
<number>

Open questions:
<list>
```

26 Final Collaboration Rule

The most important engineering rule for the remainder of the project is:

> New experimental ideas should be implemented as modules behind the common history interface rather than by repeatedly modifying the TRM recursion core.

The most important scientific rule is:

> HistoryAttention must be compared not only against vanilla TRM, but also against simpler non-attention history mechanisms.

This makes it possible to distinguish the effect of:

history access

from the effect of:

learned selective retrieval over history.

27 Immediate Next Steps

HistoryAttention Developer

1. branch from commit af02f1b;

2. read models/history/README.md;
3. design the minimal HistoryAttention mechanism;
4. implement models/history/attention.py;
5. register it in models/history/factory.py;
6. create tests/test$_history_attention.py; verify masking and zero-history behavior$;

7. run all regression tests;
8. report parameter overhead;
9. commit the implementation before running large experiments.

Non-Attention Developer

1. branch from the same commit af02f1b;
2. implement UniformMeanHistory;
3. implement RecencyWeightedHistory;
4. implement GatedHistory;
5. create shared aggregator tests;
6. preserve the same integration contract;
7. prepare controlled comparison scripts;
8. later run the same TRM-4 experimental protocol for all methods.

After both branches have validated implementations, the team should reunify the methods through the common interface and run controlled comparisons.